

M-ANT: Efficient Low-bit Group Quantization for LLMs via Mathematically Adaptive Numerical Type

Weiming Hu^{1,2}, Haoyan Zhang^{1,2}, Cong Guo³, Yu Feng¹, Renyang Guan^{1,2}, Zhendong Hua¹, Zihan Liu^{1,2},
Yue Guan^{1,2}, Minyi Guo^{1,2}, Jingwen Leng^{1,2,*}

¹Shanghai Jiao Tong University, ²Shanghai Qi Zhi Institute, ³Duke University

{weiminghu, h.y.zhang-zdy, y-feng, guanrenyang, jackeyhua, altair.liu, bonboru}@sjtu.edu.cn,
{guo-my, leng-jw}@cs.sjtu.edu.cn, cong.guo@duke.edu

Abstract—Large language models (LLMs) are one of the most important killer computer applications. The recent algorithmic advancement proposes a fine-grained group-wise quantization for LLMs, which treats a small set (e.g., 64) of values in a tensor as a compression unit. It effectively preserves the model accuracy without retraining, and has become the standard approach to efficiently deploy LLMs. On the other hand, there are works that propose various adaptive data types to better adapt to different distributions and further reduce the required bit length for LLMs. In this work, our detailed analysis unveils a key finding that while different tensors exhibit similar distributions, small groups can have markedly different distributions. As such, the group-level diversity requires a new level of adaptivity for which existing adaptive data types fail to provide.

In this paper, we propose **MANT**, a **mathematically adaptive numeric type**, featuring a more flexible encoding paradigm with a wider range of data distribution and more efficient decoding-computation fusion mechanism to address these challenges. Based on **MANT**, we develop a supporting framework to assign the appropriate data type for each group adaptively. Meanwhile, the dynamically generated Key-Value (KV) caches in LLMs introduce further complexity for real-time quantization. To tackle this, we propose an efficient real-time quantization mechanism. Besides, we implement a specific processing element (PE) to efficiently support **MANT** and incorporate a real-time quantization unit. By integrating these components into a systolic array, **MANT** unifies the group-wise weight and KV cache quantization and addresses the associated challenges. Our evaluation shows achieving, on average, $2.99\times$ (up to $4.46\times$) speedup and $2.81\times$ (up to $4.10\times$) energy reduction to the state-of-the-art LLM accelerator.

I. INTRODUCTION

In recent years, Large Language Models (LLMs) have demonstrated significant improvements in quality and accuracy across various natural language processing (NLP) tasks [9], [64], [72], [84]. However, the rapid increase in LLM parameters poses substantial challenges to memory and computational resources on existing hardware platforms like GPUs [56]–[58] and TPUs [37]. The latest Llama3 model [17], with over 405 billion parameters, requires about 800 GB of memory, exceeding the capabilities of high-end hardware platforms such as the H100 GPU [58], which has 80 GB memory. Additionally, single-batch inference involves sequentially predicting each token based on the previous ones due to the auto-regressive nature of LLMs. As a result, LLM inference mainly depends

on narrow matrix multiplications, making it memory-bound and leading to severe under-utilization of GPUs' computation resources.

Quantization has emerged as a promising method to mitigate LLMs' challenges in memory consumption and inference latency, which can be applied at different granularities such as the level of a channel [40], [43], [77] and a tensor [31], [81], [82]. Recent studies advocate for group-wise quantization, which uses a group, such as 64 contiguous elements within a channel, as the unit of quantization granularity [10], [19], [47], [50], [65], [88]. The group quantization reduces the quantization error with negligible software overhead, and hence has become the standard method for accelerating LLMs (Sec. III-A).

The above quantization methods mostly use `INT` or `FP` data types, and there are works that propose various adaptive data types to better adapt to different distributions and hence improve the quantized model performance. The adaptive data type method falls into two main categories: data-type-based and clustering-based. The former category selects data types from discrete sets based on tensor data distribution. The representative work **ANT** [31] packages several different data types for selection, including `INT` for the uniform distribution, `PoT` (Power of Two) for the Laplace distribution, and `flint` for the Gaussian distribution. The latter category utilizes clustering algorithms to generate centroids that align with the data distribution and provide considerable adaptivity. **Mokey** [82] and **GOBO** [81] exemplify this approach (Sec. III-A).

However, our analysis shows that those existing adaptive methods fail to achieve optimal performance in terms of accuracy and resource efficiency. While different tensors exhibit similar distributions, small groups can have markedly different distributions. This finding underscores the necessity for full adaptivity in group quantization to fully realize its potential. Using the per-group clustering as the ideal baseline, we show that existing data-type-based adaptive method **ANT** [31] still suffers from a significant accuracy loss (Sec. III-A).

We also show that it is not feasible to transform existing adaptive methods for the group quantization paradigm. For the adaptive data type methods, a naive approach to enhance adaptivity is to expand their set of data types. However, incorporating new data types requires additional decoders, which increase hardware costs. Additionally, compatibility

* Jingwen Leng is the corresponding author of this paper.

issues between new and existing data types may reduce computational efficiency. For instance, the NF4 data type [13] requires an FP16 MAC unit, which is incompatible with existing ANT data types. Clustering-based adaptive methods like GOBO [81] and Mokey [82] can sufficiently adapt to various distributions at the group level. However, they require codebooks for quantization and dequantization, leading to high adaptivity at the expense of encoding and computational efficiency. For instance, GOBO [81] employs the K-means algorithm to quantize weights and requires dequantization to FP16 using a codebook lookup table before computation, resulting in high adaptivity but low computational efficiency (Sec. III-B).

Finally, it is also challenging to satisfy the real-time quantization of LLMs' KV caches, which can occupy over 70% of memory and computation resources when the sequence length is sufficiently long [35], [69], [88]. As a result, the efforts on weight and activation quantization become marginal due to this dominance of KV cache. Although, in the LLM inference, KV cache behaves more like dynamically generated "weights" for the Q values in the attention mechanism [75], quantizing the KV cache at a fine-grained granularity is challenging due to substantial runtime overhead. Most previous quantization efforts [13], [14], [19], [29], [31], [47], encompassing weight-activation and weight-only quantization have sidestepped the issue of KV cache quantization (Sec. III-C).

In this study, we introduce MANT, mathematically addaptive numeric type, a novel adaptive method that aims at overcoming the above challenges, which can be summarized into three points: decoding and computation efficiency, group-wise diversity, and KV cache dynamics. To tackle the first two challenges related to efficiency and diversity, we propose MANT with a simple yet mathematical formulation to support various data types. The formulation of MANT is designed to encompass a wide spectrum of data types, theoretically extending to "infinite" variations through a smooth transition in the data type distribution. Examples of supported data types include NormalFloat [13] and flint [31]. Moreover, adopting a group-wise design approach, we enable MANT to operate at a fine-grained granularity. This inherently addresses the diversity and adaptivity requirements within LLMs. Employing group-wise formulated data types introduces computational overhead due to the diversity of data types at this fine-grained granularity. Thus, we introduce a novel computing paradigm named MANT, to mitigate this overhead significantly (Sec. IV).

To address the KV cache's dynamic challenge, we introduce a novel real-time quantization method, which lets us integrate weight and KV cache quantization into a unified framework. In our approach, we regard the KV cache as "dynamic weights", as opposed to the "static weights" of the original weights. Despite having identical tensor shapes, the K cache and V cache employ distinct computation methods, significantly complicating our design. We propose two real-time quantization techniques in two dimensions: spatial and temporal. Spatial quantization is tailored for the K cache, where all elements within a K cache group are generated simultaneously. To accommodate this, we

allocate sufficient quantization processing units for the K cache. Conversely, the V cache presents a more intricate challenge for real-time quantization. In the V cache, elements within a group are generated across the temporal dimension, meaning they emerge in different continuous iterations. Consequently, we propose a highly efficient temporal real-time quantization method tailored to the V cache (Sec. V).

Finally, MANT proposes efficient group-wise quantization for both weight and KV cache. We augment the processing element (PE) in the classic systolic array architecture to support our encoding paradigm and implement a real-time quantization engine to accelerate KV cache quantization (Sec. VI). We show that the components of MANT introduce negligible area overhead with significant speedup and energy reduction compared to the Transformer accelerators [29], [31], [43], [82].

In summary, this paper makes the following contributions.

- We analyze the challenges of group-wise quantization in both weight and KV cache, and design a new encoding paradigm to improve information utilization in group-wise quantization and inherently enable efficient decoding in LLM inference.
- We introduce a specific processing element (PE) to provide efficient encoding and computation for our encoding paradigm, and by integrating the real-time quantization components for KV cache, we implement a unified quantization system for group-wise quantization in weight and KV cache.
- We integrate these components into an existing systolic array and achieve up to $4.46\times$ speedup and $4.10\times$ energy reduction compared to existing works, with negligible area overhead.

II. BACKGROUND AND RELATED WORK

In this section, we first present the background of LLMs and their inference processes. Next, we examine quantization techniques, emphasizing group-wise quantization and commonly used data types in quantization.

A. LLM Inference Process

LLMs often adopt the Transformer architecture [75] in LLMs. Each Transformer layer incorporates an attention module and a feed-forward network (FFN). Mathematically, multi-head attention can be described as follows:

$$\begin{aligned}\text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O, \\ \text{head}_i &= \text{Attention}(HW_i^Q, HW_i^K, HW_i^V), \\ \text{Attention}(Q, K, V) &= \text{softmax}(QK^T / \sqrt{d_k})V.\end{aligned}$$

Here, W_i^Q , W_i^K , W_i^V , and W^O are parameter matrices for the i -th head and the output projection, respectively. And H is the hidden state. The *softmax* function is applied over the keys to normalize their weights, ensuring that the output is a weighted sum of the values based on the input's relevance.

The generative inference process of LLM typically involves two distinct stages: **prefill** stage and **decode** stage. The prefill stage supplies a prompt sequence to the model, establishing the context for subsequent text generation. Simultaneously, the attention operation produces K and V tensors, which are cached using the **KV cache** technique to prevent redundant

recomputation during the subsequent decode stage. The decode stage is where actual text generation occurs. In this stage, the model uses each generated token along with the KV cache to iteratively generate the next token and update the KV cache. The KV cache continues to expand until the process meets a specific termination condition. Since the **decode** stage requires only a single token as input, its operations primarily consist of matrix-vector multiplication (GEMV), resulting in a memory-bound scenario dominated by memory bandwidth.

The memory footprint of generative LLM inference comprises two major components: the weight and the KV cache, either of which can be the memory bottleneck under various scenarios [26], [35], [38], [50], [69], [88]. Compression techniques such as quantization [12], [19], [29], [31], [34], [46], [47], [81], [82], [88] and sparsity [18], [22]–[25], [27], [30], [42], [59], [76], [83], [85], [86] emerge to solve these issue.

B. Quantization Technique

The quantization technique [12], [19], [28], [65], [68], [79] is an effective method for compressing neural network parameters with minimal loss. The equation below defines the quantized data W' and the dequantized data $\hat{W}$, where s represents the scaling factor determined by the ranges of the two data types:

$$W' = \lfloor \frac{W}{s} \rfloor, \hat{W} = s \times W' \quad (1)$$

Traditional quantization methods assign mappings at the tensor or channel level, known as tensor- or channel-wise quantization. In these cases, outliers can significantly affect quantization performance by greatly increasing the rounding errors of standard values across the entire tensor or channel [12], [29].

Group-wise Quantization. To address the outlier challenge, various studies [10], [19], [47], [65], [88] advocate for group-wise quantization, which uses a group, such as 64 contiguous elements within a channel, as the unit of quantization granularity. Consider a tensor with dimensions (2048, 4096), comprising 4096 channels. With a group size of 128, each channel contains $(4096/128) = 32$ groups, resulting in a total of $2048 \times 32 = 65536$ groups across the entire tensor. Despite the modest overhead from the mapping parameter for each group, fine-grained group-wise quantization limits the impact of outliers to smaller regions, thereby enhancing performance.

Data Type for Quantization. Several studies [13], [31], [61], [82] adapt to the different distribution observed at tensor or channel levels by using customized data types. ANT [31] introduces *flint* and employs an adaptive method to select the most suitable numerical data type for each tensor from a predefined set of data types. Building on ANT, OliVe [29] develops *abfloat* to more accurately represent the distribution of outliers. Both ANT and OliVe enhance their systems with custom decoders and MAC (multiply-accumulate) units to facilitate their specific arithmetic computation workflows. Moreover, QLoRA [13] introduces *NormalFloat* (NF), which is derived from Gaussian distribution’s quantile data points, for

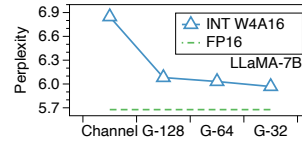


Fig. 1: LLM accuracy with different quantization granularities. We report the perplexity (PPL) metric (lower is better).

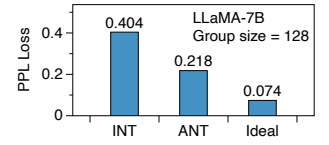


Fig. 2: Accuracy loss for INT, ANT, and Ideal (clustering algorithm K-Means) adaptive methods in group quantization.

a more precise fit. Unlike other methods, NF requires high-precision dequantization before any computational operations, as it does not directly support multiplication and accumulation. In contrast, Mokey [82] uses clustering to derive the best data type for the quantization unit. However, it requires additional codebooks to store cluster centroids. Thus, it proposes golden dictionary (GD) to mitigate those overheads. Microscaling float (MXFP) [11], [63] combines block-wise shared scale with float and INT. MXFP is similar to current group-wise quantization, but the difference is the shared scale is an 8-bit float only contains an exponent field.

C. KV Cache Optimization

Several techniques have been proposed to optimize the KV cache, including PagedAttention [41], MQA [41], GQA [2], quantization [35], [38], [50], [88], and sparsity [1], [45], [71], [85], [86]. PagedAttention is a memory management technique designed to reduce KV cache fragmentation. Several works [32], [41], [78] focus on GPU memory management to minimize the fragmentation. In GQA and MQA, multiple query heads share a single key and value head, reducing the KV cache memory footprint. Compression techniques like quantization and sparsity reduce the size or bit width of the KV cache. This paper focuses on quantization, which can also be combined with other techniques to enhance memory efficiency further.

III. MOTIVATION

In this section, we first demonstrate that the emerging paradigm of group quantization demands a high level of adaptivity, which current adaptive methods lack. We then discuss how adapting these methods to group quantization could compromise their efficiency. Given that LLMs generate KV caches during runtime, real-time quantization capability is crucial. These challenges lead to our proposal of a mathematical adaptive numerical type (MANT), which we will detail later.

A. Group Quantization Accuracy Analysis

In this subsection, we begin by comparing the accuracy of traditional channel-wise quantization with group-wise quantization [47], [50], [65], [69], [88], [88], establishing the baseline for group-wise quantization in this study. We then delve into the use of various adaptive data types in group quantization, emphasizing the necessity for full adaptivity.

Fig. 1 illustrates the perplexity when quantizing the LLaMA-7B model [72] with various granularities using the INT4-based symmetric quantization. Channel-wise quantization significantly worsens the perplexity of the examined LLM, increasing it from 5.68 to 6.85. Conversely, group-wise quantization mitigates this loss in perplexity with a group size of 128, corresponding to an average of 4.125 bits per element (16-bit scaling factor). Additionally, we observe that a smaller group size of 32 offers only a slight improvement in perplexity, but the scaling factor overhead increases by $4\times$.

Given this analysis, we adopt a group size of 128 as our standard configuration for the remainder of this section. Previous research indicates that the INT data type is not optimal for accuracy since tensors or channels exhibit varied distributions, leading to the proposal of various adaptive data types [29], [31], [81], [82]. We evaluate their efficacy in the context of group quantization, which falls into two main categories: data-type-based and clustering-based.

Data-type-based adaptive methods select data types from discrete sets based on tensor data distribution. ANT [31] is a representative example of the data-type-based method. ANT packages several different data types for selection, including INT for the uniform distribution, POT (Power of Two) for the Laplace distribution, and flint for the Gaussian distribution.

Clustering-based adaptive methods utilize clustering algorithms to generate centroids that align with the data distribution and provide considerable adaptivity. Mokey [82] and GOBO [81] exemplify this approach, though they focus on tensor- or channel-wise quantization. In our study, we adapt them to group quantization through per-group clustering.

Fig. 2 compares the accuracy of the methods described above for the LLaMA-7B model under 4-bit group-wise quantization. The group-wise ANT method outperforms the INT type by dynamically selecting from three data types to better match the value distribution, resulting in reduced perplexity (PPL) loss. Moreover, per-group clustering adjusts more effectively to the value distribution of each group, establishing itself as the accuracy-optimal and ideal adaptive method. This approach achieves nearly lossless 4-bit quantization, equivalent to 16 centroids per group. However, this ideal scenario is impractical due to the significant overhead associated with storing per-group centroids, effectively rendering it a 6-bit quantization.

To illustrate the group-wise diversity in data distribution, we sampled the weights of the Q and V tensors in LLaMA-7B model. We normalized all sampled data to their absolute maximum values, which ranged from -1 to 1. Fig. 3 displays the cumulative distribution function (CDF) for the tensor, channel, and group levels, respectively. We observed that the diversity at the group level is significantly higher than at the tensor level. In simpler terms, while different tensors exhibit similar distributions, groups can have markedly different distributions. This finding underscores the necessity for full adaptivity in group quantization to fully realize its potential.

Takeaway 1. The group quantization is an emerging paradigm to accelerate LLMs, and the significant group-level diversity

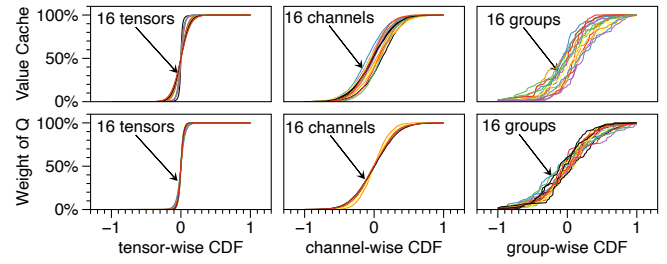


Fig. 3: The cumulative distribution function (CDF) of the tensor, channel, and group, respectively. The tensor data were taken from layers 8 to 23, while the 16 channel and group data were sampled from one tensor with specific strides.

requires a high level of adaptivity to fully unleash its potential.

B. Group Quantization Efficiency Analysis

In this subsection, we provide a detailed efficiency analysis for the above adaptive quantization methods. In Tbl. I, we compare OliVe [29], ANT [31], GOBO [81], and Mokey [82] with INT regarding the efficiency of computation, encoding, and decoding. In this paper, we use the term encoding (decoding) interchangeably with quantization (dequantization).

Data-type-based adaptive methods such as ANT [31] and Olive [29] achieve computational efficiency comparable to INT. Both utilize specialized decoders that decode these data types prior to computation, resulting in high decoding efficiency. However, as previously demonstrated, these methods suffer from limited adaptivity in the group quantization paradigm. A straightforward approach to enhance adaptivity is to expand their set of data types. However, incorporating new data types necessitates additional decoders, escalating hardware design costs. Additionally, compatibility issues between new and existing data types may reduce computational efficiency. For instance, the NF4 data type [13] requires an FP16 MAC unit, which is incompatible with existing ANT data types.

Takeaway 2. Enhancing the data-type-based adaptive method for group quantization is challenging and requires a careful balance for the computation and decoding efficiency.

Clustering-based adaptive methods like GOBO [81] and Mokey [82] can sufficiently adapt to various distributions at the group level. However, they require codebooks for quantization and dequantization, leading to high adaptivity at the expense of encoding and computational efficiency. For instance, a 16-entry codebook with 8 bits per entry requires 128 bits per group, creating an inevitable trade-off between adaptivity and memory overhead. GOBO [81] employs the K-means algorithm to quantize weights and requires dequantization to FP16 using a codebook lookup table before computation, resulting in high adaptivity but low computational efficiency. Conversely, Mokey [82] enhances the computation of clustering-based methods by using indices for centroid values via approximate calculations, though matrix multiplication still relies on floating-point units, increasing overhead compared to integer units. Furthermore, Mokey creates one golden dictionary for

TABLE I: Features of DNN accelerators with adaptive and flexible data types are summarized. Here, ‘Effi.’ stands for efficiency, ‘Med.’ for medium, and ‘LUT’ for lookup table.

Architecture	Encode		Computation			Decode		Adaptivity
	Method	Effi.	Method	Bit	Effi.	Method	Effi.	
INT	Round	High	INT	4 & 8	High	Calculation	High	Low
OliVe [29]	Search	Med.	INT	4 & 8	High	Decoder	High	Med.
ANT [31]	Search	Med.	INT	4 & 8	High	Decoder	High	Med.
Mokey [82]	Cluster	Med.	Float	4 & 8	Med.	Calculation	Med.	Low
GOBO [81]	Cluster	Low	Float	16	Low	LUT	Med.	High
MANT	Search	Med.	INT	4 & 8	High	Calculation	High	High
	Map	High						

all activations and weights, akin to using a single data type in quantization, thus reducing adaptivity.

Takeaway 3. Deploying the clustering-based adaptive methods under group quantization is challenging owing to the low encoding and computation efficiency.

C. Support for Real-time Quantization

The above group-wise diversity presents a challenge for both weights and KV cache. In addition, KV cache faces challenges in real-time group-wise quantization because the KV cache is generated dynamically during LLM inference.

To facilitate low-precision computation in group-wise quantization, it is necessary to quantize K and V along the inner dimension. This requirement stems from the support for matrix inner product operations in most GPUs and TPUs. During these operations, the group-wise scaling factor can be extracted from the multiply-accumulate process. Fig. 4 depicts the computation process of K and V during the decode stage. We define the dimension used for matrix inner product operations as the inner dimension. The inner dimensions of the K and V caches differ; the K cache requires a transpose operation, whereas the V cache does not, complicating the situation.

In the prefill stage, K and V can easily compute the scaling factor for each group. During the decode stage, the newly generated K vector is concatenated along the inner dimension of the K cache, enabling immediate quantization. However, the newly generated V vector is associated with different groups, with only one element per group produced per iteration. This process prevents the scaling factor for the entire group from being obtained in a single iteration, posing a significant challenge for the real-time quantization of the V cache.

Given those challenges, we propose MANT with a mathematical encoding format that can fuse with integer computation and enhance the decoding efficiency. In addition, this encoding format provides sufficient adaptivity for group-wise quantization. Regarding the challenge in KV cache, MANT employs a real-time quantization engine that ensures efficient encoding and decoding for KV cache. By addressing these challenges, MANT enables efficient low-bit group-wise quantization.

IV. MATHEMATICALLY ADAPTIVE NUMERICAL TYPE

We propose MANT (mathematically adaptive numerical type), a directly computable and adaptive data representation that achieves efficient encoding/decoding under the group-wise quantization framework. We first present the formal mapping

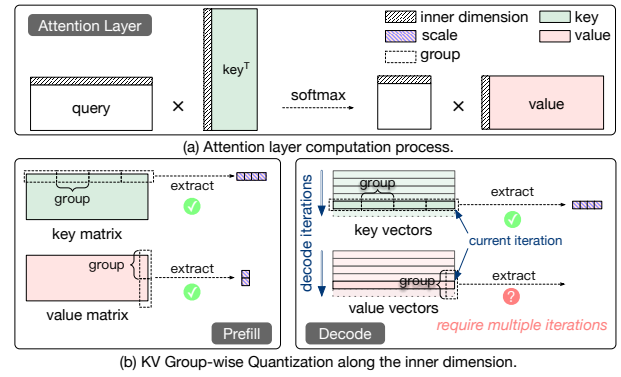


Fig. 4: Comparison of group-wise K and V cache quantization. They have different inner dimensions due to the transposition of K (key).

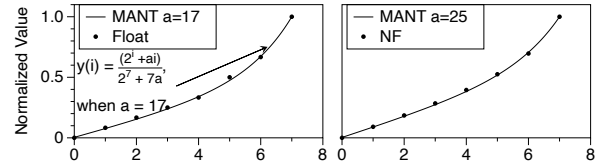


Fig. 5: Using different a in MANT for data type approximation.

of MANT-encoded INT values in Sec. IV-A. We then describe its efficient encoding process in Sec. IV-B and how it eliminates the necessity for a data type-specific decoder in Sec. IV-C.

A. Mapping Representation

Two key considerations drive the design of our mapping representation. The first consideration is the ability to accommodate the diverse distributions of group-wise data. The second consideration is the computation efficiency, suggesting a direct computation approach using the INT value. Based on the two considerations, we propose a mapping representation defined by the following equation:

$$Value_{grid} = \pm(a \times |INT| + 2^{|INT|}) \quad (2)$$

In Equation (2), the $Value_{grid}$ is the quantization grid, a is a group-wise constant, and INT is a sign-magnitude representation of INT4 within the context of symmetric quantization, which covers the range from $[-7, 7]$. We take INT4 as an example to explain our design since the computation of INT4 is easy to support and INT4 is memory-aligned. The quantization grid, denoted as $Value_{grid}$, is constructed as follows: $\{\pm(a \times 0 + 2^0), \pm(a \times 1 + 2^1), \dots, \pm(a \times 7 + 2^7)\}$.

Our core idea is to approximate each data type by changing the coefficient a . The following equation defines the positive value function of different data types.

$$y_{INT}(i) = i, y_{PoT}(i) = 2^i, y(i)_{MANT} = ai + 2^i, i \in [0, 7] \quad (3)$$

$$y_{NF}(i) = \Phi^{-1}\left(\frac{i \times (1 - \epsilon) \times 0.5}{7} + 0.5\right), \quad i \in [0, 7]$$

Here, integer i ranges from 0 to 7, and Φ^{-1} represents the probit function, the inverse of the cumulative Gaussian distribution. The small ϵ prevents Φ^{-1} from reaching ∞ . To represent a given

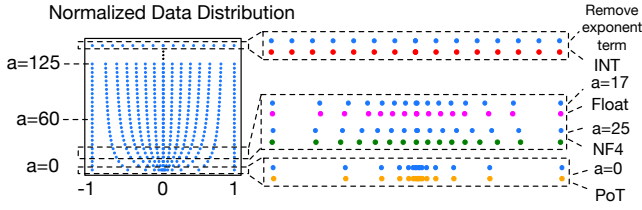


Fig. 6: The diversity of data representation with coefficient a . The 4-bit data types have 16 points. Left part of figure shows how the distribution varies with the increase of a . Right part of figure shows that when setting a to a different value, it can nearly match the distribution of several data types, including float, NF, PoT, and INT.

datatype in MANT, we only need to find a proper coefficient a that minimizes the approximation error. For example, to represent $y_{\text{INT}}(i)$, the goal is to minimize the target function $\argmin_a(|\frac{i}{7} - \frac{ai+2^i}{7a+2^7}|)$, where the division by 7 and $7a+2^7$ serves to normalize and match the maximum values of the distributions. Fig. 5 shows the values of a to represent Float and NF.

Fig. 6 shows the normalized data distribution across various coefficient a values. Modifying the coefficient a leads to a smooth change in the data distribution, allowing for a versatile mapping representation that accommodates a wide range of data types. For instance, setting the coefficient a to 0 transforms the mapping representation to $\text{Value} = \pm(2^{\lfloor \text{INT} \rfloor})$, it makes MANT exactly match the data type PoT [53], [90]. Besides, MANT can approximate the distribution of float and NormalFloat (NF) [13] when setting coefficient a to 17 and 25, respectively.

It is worth noting that the role of coefficient a is to change the data distribution. Meanwhile, coefficient a can be integrated into the computing process of dequantization with low computation overhead, which we detail later. Our experiment finds that the variation of the data distribution becomes marginal when the coefficient a exceeds 128. As such, we constrain the data range of a within 128, allowing 8-bit encoding for a .

B. Encode

Based on the mapping of MANT, we describe its encoding process, assuming that activations are quantized to 8 bits and weights to 4 bits with a determined coefficient a from original FP16 values. In the group-wise quantization, each group stores the metadata that includes both the scaling factor and the coefficient a . The quantization process is defined as:

$$s_X = \frac{\max(|X_{\text{FP16}}|)}{\max(\text{INT8})}, s_W = \frac{\max(|W_{\text{FP16}}|)}{\max(W_{\text{grid}})} \quad (4)$$

$$X_{\text{INT8}} = \lfloor \frac{X_{\text{FP16}}}{s_X} \rfloor, W_{\text{INT4}} = \argmin(\frac{W_{\text{FP16}}}{s_W} - W_{\text{grid}})$$

X represents the activation and W represents the weight. s_X is the scaling factor of activations and s_W is the scaling factor of weights, while W_{grid} is the quantization grid with specific a . Fig. 7 shows an example of MANT with $a = 17$. The rounding and encoding process of weights in Fig. 7 corresponds to $\argmin$ operation in the Equation (4). The encoding process is

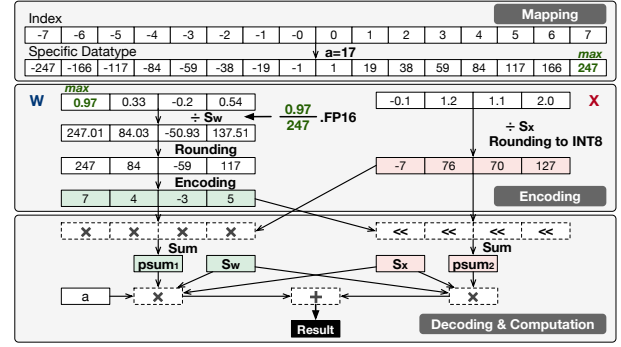


Fig. 7: The encoding and decoding process of MANT.

expensive since it necessitates finding the nearest point within a non-uniform grid. Nonetheless, the weights encoding process can be done offline, avoiding the runtime overhead.

C. Decode & Compute Fusion

We use the example in Fig. 7 to illustrate the fused decoding and computing process. The primary advantage of MANT lies in its ability to perform computations efficiently in low-bit formats, with the decoding process integrated into matrix multiplication. In our approach, the activations and weights are quantized along their cumulative dimensions, allowing to decouple the computation of the scaling factors s_X and s_W from multiplication and addition. The combined decoding and computing process is described by the equation below:

$$\begin{aligned} \hat{X}_{\text{FP16}} \times \hat{W}_{\text{FP16}} &= (X_{\text{INT8}} \times s_X) \times (W_{\text{grid}} \times s_W) \\ &= [X_{\text{INT8}} \times W_{\text{grid}}] \times s_X s_W \\ &= [X_{\text{INT8}} \times (a \times W_{\text{INT4}} + 2^{W_{\text{INT4}}})] \times s_X s_W \\ &= \underbrace{[X_{\text{INT8}} \times W_{\text{INT4}}]}_{\text{psum}_1} \times a \cdot s_X s_W + \underbrace{[X_{\text{INT8}} \times 2^{W_{\text{INT4}}}] \times s_X s_W}_{\text{psum}_2} \end{aligned} \quad (5)$$

$\hat{X}_{\text{FP16}}$ and $\hat{W}_{\text{FP16}}$ are dequantized activations and weights, respectively. In Equation (5), W_{grid} is derived from Equation (2). Thus, the computation can be divided into integer multiplication and integer shift operations. Moreover, it facilitates computations in a mixed-precision mode, using 8-bit activations and 4-bit weights, eliminating the need to dequantize low-bit weights before computation. To simplify the discussion, we omit the details of handling the sign bit in Equation (5), which can be efficiently processed in hardware.

V. MANT QUANTIZATION FRAMEWORK

This section introduces the MANT quantization framework, focusing on the quantization methods for weight, activation, and KV cache. Initially, we discuss the weight quantization with MANT in Sec. V-A. Subsequently, we illustrate why activation is quantized with INT8 in Sec. V-B. Finally, we introduce a real-time MANT quantization mechanism to tackle the challenge brought by the dynamic KV cache and customize quantization strategies for K and V cache, respectively, in Sec. V-C.

A. Weight Quantization

Weight is encoded offline to select the most suitable coefficient a for each group. Within the MANT framework, a calibration dataset [20] is used to identify the coefficient a that minimizes the mean square error (MSE) of the output. The following equation describes the optimization objective:

$$a = \underset{a}{\operatorname{argmin}} \|X\hat{W}_a - XW\|_2^2 \quad (6)$$

$\hat{W}_a$ represents the weight that is first quantized and subsequently dequantized using a specific coefficient a . Broadening the range of data types in the search space typically enhances accuracy. Nevertheless, slight modifications to a , increasing or decreasing it by one, only slightly alters the data distribution. Consequently, we selected 16 data types for weight quantization, including the set $\{0, 5, 10, 17, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120\}$ and an additional INT option.

B. Activation Quantization

For the activation quantization, we follow the common practice that quantizes them using the group-wise INT8 for near-lossless accuracy and efficient hardware implementation [19], [40], [47]. Meanwhile, unlike weight and KV cache, activations are temporary variables. They are dynamically generated, quantized, and consumed, so they cannot be reused in the latter inference iteration. Finally, the weights and KV caches already consume most of memory in the LLM inference so that activations occupy only a minor fraction of the memory (<5% [80]). As such, further reducing the activation bit width only leads to marginal improvement.

This choice of using INT8 to activation quantization also allows the computation between the weight that quantized using our MANT and the activation that quantized using INT8 to exploit the low-bit computation units, as detailed in Sec. IV-C. MANT further leverages a streaming comparator unit to determine the maximum value of activations, facilitating the activation quantization.

Real-time quantizing activation requires two operations: derive the maximum value of a group to calculate scaling factor and map the FP16 value to INT8. MANT hides the latency of searching the maximum value with hardware design fusing this operation into the streaming of systolic array output, as detailed in Sec. VI-C.

C. KV Cache Quantization

The Quantization Dimension. We quantize K and V along the accumulation dimension, as we discuss in Fig. 4. Some prior algorithmic works [35], [50] do use a different quantization direction for V cache, which we believe is orthogonal to our work. First, MANT can also work with the channel direction of V cache quantization, which may sacrifice the computation efficiency of all quantization methods, including ANT. Second, those prior works are based on channel-level quantization or extremely low bit, while the impact of directions for the 4-bit group-level quantization is much smaller, as we show later. Finally, emerging incoherent processing algorithms [3], [74],

[77] (where SmoothQuant [77] is a special case) are very promising to further mitigate this gap.

Select MANT through Variance. In KV cache quantization, real-time data type selection is required. Although the searching method based on MSE achieves less accuracy loss, it requires performing quantization to each data type for MSE searching, which is intolerable in a real-time scenario. Thus, we develop a mapping mechanism based on the data characteristics like variance, which can be derived in a streaming way.

Since variance can reflect the distribution of a data group and MANT with different a has a different variance, MANT determines the coefficient a using a variance. Then, calculate the variance:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n x_i^2 - \left(\frac{1}{n} \sum_{i=1}^n x_i \right)^2 \quad (7)$$

In MANT, a larger a corresponds to higher variance, with each value of a associated with a specific variance range. We first sample the K and V tensors through a calibration dataset [20], and select a for each group to minimize quantization error. Next, we calculate the variance of the groups with different a to decide the appropriate range. The elements within each group are first normalized, ensuring that the absolute maximum value in the group is scaled to 1. Following this normalization step, the variance of the normalized elements is then computed. For example, when $a = 35$, the variance is 0.104; when $a = 45$, the variance is 0.118. We define the range for $a = 40$ as [0.104, 0.118]. If the variance of normalized data falls within a specific range of a , it will be quantized with that a with MANT. We fuse the computation flow of variance with matrix multiplication to hide latency, which we will detail in Sec. VI-C.

Prefill Stage. During the prefill stage, the input comprises a sequence, so the K and V are both matrices, where the sequence length typically exceeds the group size. Thus, both the K cache and V cache can obtain the data needed to calculate variance. By selecting the appropriate a based on the variance, the K cache and V cache can be quantized to 4-bit MANT.

Decode Stage. In the decode stage, since inputs are vectors, generated K and V are vectors. Thus, the K cache can obtain all data of the group in a single iteration to execute real-time quantization, similar to activation in decode stage. The difference is that each group needs to calculate the partial sum of x_i and x_i^2 as well as the maximum value since K cache needs to be quantized to 4-bit MANT. However, when it comes to V cache, a new challenge arises as each iteration only generates one element of a group.

To address this, we propose a two-phase quantization scheme for the V cache, as shown in Fig. 8. We define every G iterations in the decode phase as a process window for V cache, where G is the group size. In the first phase, newly generated V vector is quantized to INT8 with channel-wise scaling factors derived from prefill stage, denoted as ‘scales’ in Fig. 8. Meanwhile, we update the maximum value and partial sum of v_i and v_i^2 , denoting the parameter in a group as v_i . This operation persists until the process window is full.

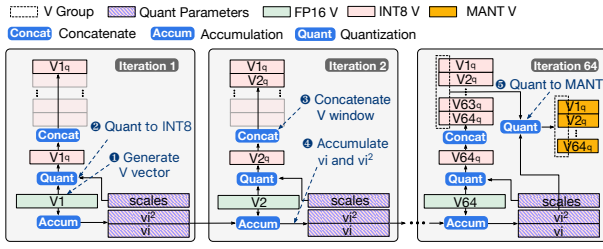


Fig. 8: The update process of V cache with a group size of 64. Each iteration generates a new V vector, quantizes it to INT8, and accumulates the parameters for MANT quantization. In the 64th iteration, the V cache is quantized to 4-bit MANT.

The second phase involves quantizing the 8-bit V cache to 4-bit MANT. When the process window is full, we calculate the variance with partial sum of v_i and v_i^2 by (7). Then, the coefficient a is decided similarly to the prefill phase, and the stacked INT8 V cache is quantized to 4-bit MANT.

This two-phase quantization scheme effectively quantizes all but the latest V vectors in the processing window to 4 bits. The process window is similar to the residual group used in KIVI [50]. The overhead of INT8 operation of V cache in processing window is marginal and tolerable. Besides, it helps improve the quality of newly generated tokens, since some studies have shown that the latest tokens are more important [16].

VI. MANT MICROARCHITECTURE

This section outlines the microarchitectural details of MANT that enable efficient quantization for LLM inference. Our hardware design supports the fusion of MANT decode and computation, as detailed in Sec. IV. Additionally, it integrates microarchitectural components that facilitate real-time quantization of activations and the KV cache, as described in Sec. V.

A. Accelerator Overview

We first explain the rationale for the architectural extension. Initially, the computing paradigm of MANT introduces an additional term in matrix operations, which the current Processing Element (PE) does not directly support. Secondly, we need an on-chip quantization engine to minimize real-time quantization delays for activation and KV cache. Finally, our quantization framework uses varying bit widths for weights, activations, and the KV cache, necessitating mixed-precision computations.

Fig. 9 provides an architectural overview that adopts a weight-stationary systolic array with only slight modifications. The array consists of 32×32 units, each capable of handling 8-bit integers, referred to in this section as Processing Elements Group (PEG). The real-time update engine incorporates a lightweight comparator unit to identify maximum values. During dequantization, output values are multiplied by a scaling factor and processed through this comparator to determine maximum values for use in the quantization units.

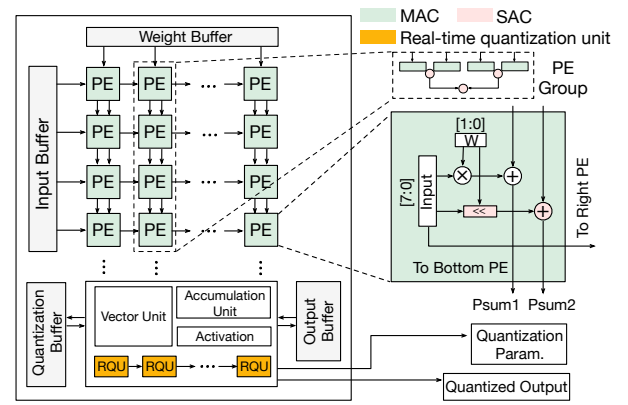


Fig. 9: Accelerator overview. MAC and SAC are multiply-accumulate and shift-accumulate units, respectively.

B. MANT PE Unit

Tile Architecture. We begin by describing the tile architecture of MANT, which is based on a weight-stationary systolic array [37]. Inputs to the systolic array are fed from the left, and outputs exit from the bottom. The array's configuration varies with the type of operation: a 32×32 array for $\text{INT8} \times \text{INT8}$ operations, a 64×32 array for $\text{INT8} \times \text{INT4}$ operations, and a 128×32 array for $\text{INT8} \times \text{INT2}$ operations. This variation in configuration is due to the architecture's support for mixed-precision operations. Specifically, the bit width of weight can be configured to 2, 4, or 8 bits.

Our discussion will focus on INT8 and INT4 operations, where the weight tile dimension is $(64, 32)$, the input tile dimension is $(m, 64)$, and the output tile dimension is $(m, 32)$. In this configuration, each column of PEs represents the cumulative dimension, with elements in a column sharing both a scaling factor and the coefficient a of MANT.

Each PE in our architecture includes both multiply-accumulate (MAC) and shift-accumulate (SAC) components. The MAC components manage the first partial sum ($psum_1$) defined in Equation (5), while the SAC handles the second partial sum ($psum_2$). Executing $psum_1 \times a$ within the PE facilitates the addition of $psum_1$ and $psum_2$. However, this process introduces an additional multiplication step across all PEs. To streamline computations in the systolic array, MANT directs $psum_2$ through an additional lane.

Mixed Precision. Given the varying bit-width requirements for weight, activation, and KV cache quantization, support for mixed precision is crucial, as noted in previous studies [6], [7], [29], [31], [52], [60], [62], [70], [91]. Our approach aligns with methodologies found in BitFusion [67]. As depicted in Fig. 9, each Processing Elements Group (PEG) within our system includes four Processing Elements (PEs), each capable of handling computations between INT8 and INT2 formats. Two PEs combined can perform an $\text{INT8} \times \text{INT4}$ operation. A single PEG can execute either one $\text{INT8} \times \text{INT8}$ operation or four $\text{INT8} \times \text{INT2}$ operations within one cycle. This configuration efficiently supports mixed-precision computations

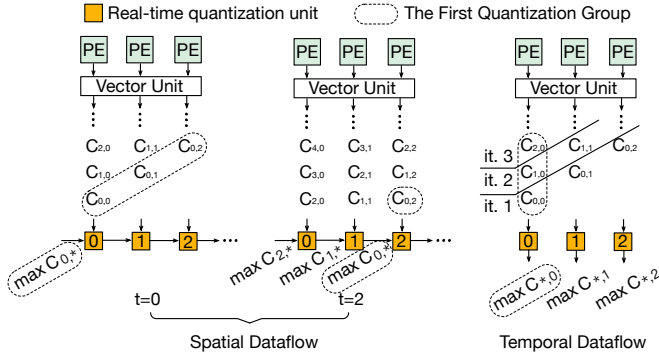


Fig. 10: Spatial and temporal dataflow mode of real-time quantization unit. “It.1” in this figure means the first iteration for V group during the decode stage of LLM inference.

across 2, 4, and 8-bit widths for weights and KV cache.

C. Real-time Quantization Unit

We describe how our architecture implements real-time quantization for activations, K cache, and V cache. We leverage the dataflow [8], [33], [37], [49], [92], [93] of matrix multiplication to hide latency. In this scenario, the group size is set to 64.

Our real-time quantization unit (RQU) consists of two components: an FP16 comparator and two FP16 accumulators. In MANT, we utilize 32 RQUs matching the size of the systolic array to determine the maximum values and calculate variance parameters for the K cache and V cache. Besides, it can derive the maximum values for INT activation quantization. The RQU supports both **spatial** and **temporal** dataflow modes. Fig. 10 illustrates these two modes and shows an example where RQUs are used to compute their maximum values. Once the systolic array completes the final computation in the accumulation dimension, it captures the full results of the output tile, which are then quantized to a low-bit format. In the weight-stationary systolic array, the leftmost column initiates the computation, and the rightmost column begins 31 cycles later, offering a chance to pipeline the process of maximum comparison with the preceding computations.

As depicted in Fig. 10, at $t = 0$, the first RQU (RQU_0) uses $C_{0,0}$ as the initial maximum value, $\max(C_{0,*})$, and passes it to RQU_1 . At $t = 1$, RQU_1 compares the value of $C_{0,1}$ with $\max(C_{0,*})$ and forwards the greater value to RQU_2 . By $t = 32$, RQU_{31} outputs the final maximum value $\max(C_{0,*})$. From this point, the RQUs operate in a fully pipelined manner, with RQU_{31} producing one maximum value per cycle. Given that the output tile’s dimensions are $(m, 32)$ (with $m = 1$ during the decode phase), identifying the maximum of 64 elements in one group requires two comparison rounds. Subsequently, activation can be quantized based on their maximum values.

Conversely, the V cache utilizes the **temporal** dataflow mode of RQU, as illustrated on the right side of Fig. 10. In this mode, each RQU compares values from the same column of the systolic array and retains the maximum value in its register.

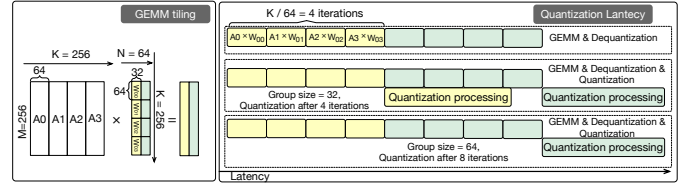


Fig. 11: An example of GEMM tiling and the corresponding quantization overhead.

During the prefill phase, each RQU captures the maximum value of every 64 elements, allowing the 64 elements from the same column to be quantized based on these maximum values.

The dataflow of calculating sum and square sum for variance is similar to deriving maximum value, except that the FP16 accumulator replaces the comparator. This process of real-time quantization is pipelined with the dataflow of systolic array. Therefore, when the last computational dataflow of the systolic array finishes, it only needs several cycles to dequantize and compute the quantization parameter through the RQU. The RQU improves the efficiency of real-time quantization for activations and the KV cache.

D. Buffers

We use multi-bank structures for input, weight, output, and quantization buffers to enable parallel access. Fig. 10 shows that the 32 outputs from different rows and columns are dequantized in one cycle. It requires 32 scaling factors for activations (s_X), 32 for weights (s_W), and 32 a . The s_W and a are fixed, so the key point is to ensure that s_X are stored in different banks of quantization buffer to prevent bank conflict.

E. GEMM Computation

We explain how our MANT-based accelerator design fuses the (de)quantization process with the original GEMM computation to hide their overhead.

Dequantization. The dequantization process of a group-quantized tensor is to multiply each element with its scaling factor. Typically, as shown in Fig. 11, the $M \times K \times N$ GEMM computation is tiled to run on the PE array. As long as the group quantization size is greater than the PE array’s accumulation dimension (i.e., the K dimension), the scaling factor multiplication can be deferred to after the output of PE array and before the accumulation of the partial sum since all elements within the array share the same scaling factor. As such, we can concurrently compute the scaling factor product using the vector unit when the PE array starts computation, and augment the original accumulator design with additional multipliers. Since this accumulation is fully pipelined, its impact on the overall latency is negligible.

Quantization. The quantization only occurs when all partial sums finish the accumulation to the global sum. In Fig. 11, after four iterations along the K dimension, the global sum and maximum value for the output tile are determined. The vector units then compute the scaling factor based on the maximum

value and perform division for that output tile. Note that the first division for the scaling factor occurs once for an entire group, and the second division occurs for each element. Fig. 11 shows that quantization latency can be hidden by overlapping with GEMM tile computation. In our design, we model a 12-cycle non-pipelined division unit, which requires 12 K -dimension iterations to completely hide this latency. We show that this has little impact on our overall performance. In summary, MANT implements the operation fusion [55], [87], [89], [92] between decoding, GEMM, dequantization, and quantization.

VII. EVALUATION

We implement the MANT quantization framework and compare its accuracy with four baselines: Tender [43], OliVe [29], ANT [31], and BitFusion [67]. Subsequently, we conduct a detailed analysis of these accelerators' area, performance, and energy consumption, focusing on both the linear and attention layers. Based on this analysis, we evaluate the performance across various sequence lengths. Finally, we evaluate the results by employing group-wise quantization in our baselines.

A. Experimental Setup

Models and Datasets. We evaluate MANT on three families of Large Language Models (LLMs), including LLaMA-1 7-65B [72], LLaMA-2 7B [73], LLaMA-2 13B, OPT-6.7B [84] and OPT-13B. Our evaluation covers different model architectures and sizes. We evaluate the quantization results using the zero-shot task on the Wikitext dataset (wikitext-2 version) [51], with perplexity (PPL) of the generated sentences serving as the metric. The lower PPL indicates the better results. We take two subsets from the Pile dataset [20] as the calibration dataset to avoid a biased result. The OPT and LLaMA models are evaluated by lm-eval-harness framework [21].

We employ MANT on generation tasks to evaluate the KV cache quantization in decode stage. We use TruthfulQA [48] on lm-eval-harness for normal context evaluation and TriviaQA [36] on LongBench [4] for long context evaluation.

Quantization Details. We evaluated our approach against three baselines of Transformer quantization accelerators Tender [43], OliVe [29], ANT [31] in 4-bit and 8-bit PTQ (Post-Training Quantization) settings. All of these are open-source frameworks. ANT performs activation-weight quantization across CNN and Transformer models, such as BERT [15], and we extended its framework to support LLM quantization. Tender and OliVe is the state-of-the-art (SOTA) LLM accelerator that mitigates outliers with a hardware-friendly methodology.

ANT and OliVe apply tensor-wise quantization to activations and channel-wise to weights. Tender divides activation channels into several chunks. In each chunk, Tender further reorders and packages the channels into groups so that the scaling factors of the adjacent groups can be derived via a simple 1-bit shift. By moving the shift to accumulation, Tender accommodates outliers with a unified scaling factor in a chunk. This method is orthogonal to MANT and can work together. We evaluate Tender here as a SOTA LLM accelerator. Since Tender, OliVe, and

ANT do not quantize the attention layer of LLMs, our analysis of their quantization accuracy is limited to linear layers.

MANT is configured with a group size of 64, applying this setting to quantize activations, weights, and KV cache. MANT employs INT for activation quantization. For weight and KV cache quantization, MANT selects the most appropriate encoding from the set $a = \{0, 5, 10, 17, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120\}$ and the INT. MANT selects the coefficient a for KV cache based on the data variance, as described in Sec. V-C.

Implementation. We evaluate the performance and energy consumption of MANT compared to baseline accelerators. The MANT's processing elements (PE) and comparator are implemented in Verilog. They are synthesized using Synopsys' Design Compiler (ver. O-2018.06-SP1) with the TSMC 28nm standard cell library to obtain power and area statistics. Additionally, the SRAM module's power and area statistics are simulated using CACTI [54].

We develop a simulator based on the cycle-level simulator DNNWeaver [66] to evaluate performance. ANT, OliVe, and BitFusion, which are built on this simulator, all support mixed precision. We also implement the architecture of Tender in this simulator and support both INT4 and INT8 MAC. ANT has also extended its design using a GPU simulator [5], [39], [44], but our evaluation focuses exclusively on ASIC implementations.

Accelerator Details. BitFusion is the baseline mixed-precision architecture. ANT is a mixed-precision architecture with adaptive data types. OliVe further enhances this by supporting outlier-victim pairs with adaptive data types and mixed precision. Tender can be easily extended to support mixed precision. Consequently, we consider these architectures as baselines in our evaluation.

For a fair comparison, in the evaluation of the linear layer, we employ the mixed-precision in Tender and OliVe to align the PPL results. OliVe and Tender utilized 4-8 mixed precision. The 8-bit ANT can not align the PPL results either, so we label it ANT*. The 8-bit ANT does not adaptively select the data type and only uses INT, so we can view ANT* as a coarse-grained INT8 quantization.

In addition, we configure the number of PEs for each accelerator based on their area overhead. We aim to compare their performance and power consumption while ensuring nearly equivalent area and PPL. We set the same configuration for memory bandwidth, on-chip buffer size, and frequency across all accelerators. In linear layer, the sequence length is set to 2048, with a batch size of 1.

Tender, OliVe, and ANT do not quantize the attention layer in their evaluations. Therefore, the statistics for these baselines in the attention layer are the same, as shown in Fig. 13.

B. Accuracy Evaluation

Results in the Linear Layer. In Tbl. II, we present the PTQ results for the discussed methods. Both ANT and OliVe implement mixed-precision quantization, but their hardware

TABLE II: The PTQ results of Wikitext on various LLMs, the metric is perplexity, and the lower is better. “Act.” is the activation, “W” is the weight, and “Atten.” is the attention layer. MANT employs group-wise quantization. ANT, OliVe, and Tender use tensor/channel-wise quantization.

Method	Linear (bit)		Atten. (bit)		LLaMA 1&2 (Perplexity)						OPT 6.7B	OPT 13B
	Act.	W	Act.	KV	1-7B	1-13B	1-30B	1-65B	2-7B	2-13B		
FP16	16	16	16	16	5.68	5.09	4.10	3.53	5.47	4.88	10.86	10.13
ANT	4	4	16	16	61.35	53.74	19.31	21.57	124.18	333.29	6.4E+3	9.6E+3
OliVe	4	4	16	16	32.15	15.64	13.59	12.85	44.07	50.28	39.18	65.42
Tender	4	4	16	16	23.85	13.68	12.07	8.85	36.47	55.08	13.56	16.43
MANT	4	4	16	16	6.09	5.38	4.42	3.83	5.92	5.24	11.29	10.62
ANT	8	8	16	16	9.50	8.46	6.70	5.34	10.68	16.11	14.60	165.9
OliVe	8	8	16	16	5.86	5.28	4.37	3.80	5.73	5.06	11.24	10.49
Tender	8	8	16	16	5.87	5.28	4.27	3.74	5.77	5.09	10.93	10.17
MANT	8	4	16	16	5.79	5.20	4.18	3.61	5.57	4.96	10.98	10.18
MANT	8	4	8	4	5.97	5.31	4.37	3.75	5.79	5.14	11.14	10.31

TABLE III: Evaluation on generation tasks TruthfulQA and TriviaQA. The metric used is the BLEU score for TruthfulQA and the F1 score for TriviaQA. Higher is better.

Tasks	LLaMA-2-7B			
Weight & Act.	FP16	W4A8		
KV cache	FP16	FP16	INT4	4-bit MANT
TruthfulQA	27.88	27.55	25.48	26.19
TriviaQA	87.72	86.38	85.13	86.86

design requires that activations and weights share the same bit width. Thus, we evaluate the quantization under 4-bit weight-activation (W4A4) and 8-bit weight-activation (W8A8) settings. In W4A4 scenario, MANT shows better PPL results than other methods. MANT with W4A8 achieves under 0.11 PPL loss in LLaMA family models and under 0.12 PPL loss in OPT models.

ANT and OliVe with W4A4 demonstrate significant PPL loss due to the sensitivity of activations, but OliVe’s W8A8 configuration shows promising results. Tender’s W4A4 is better than ANT and OliVe because its progressive strategies in the chunk offer a wider region of representation. Tender’s W8A8 result is comparable to OliVe’s W8A8. In OPT models, Tender’s W8A8 surpasses MANT’s W4A8, while MANT’s W4A8 achieves the best perplexity results in the LLaMA family models.

Quantize both the Linear and Attention Layer. The Wikitext task only involves the prefill stage, so Tbl. II reflects the results of quantizing the KV cache during this stage. Results show that MANT achieves a PPL loss below 0.3 when quantizing both linear and attention layers. Compared to quantizing only the linear layer, quantizing the attention layer with MANT results in a PPL loss below 0.2. Quantizing the KV cache along the opposite dimension [35], [50] further reduces the loss to 0.1, but performing the low-bit computation in this way is challenging.

Analysis for Generation Tasks. For a comprehensive analysis of KV cache quantization, we evaluate MANT on generation tasks TruthfulQA and TriviaQA. In Tbl. III, W4A8 indicates that weights are quantized to 4-bit using MANT, and activations are quantized to INT8. For KV cache quantization, MANT outperforms INT and achieves a loss below 1.7%.

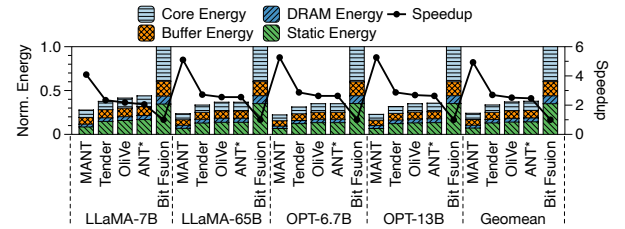


Fig. 12: Speedup and energy breakdown in the linear layer on accelerators. The performance and power consumption of accelerators are evaluated under the nearly equivalent area and PPL except ANT, so it is denoted as ANT*.

C. Performance, Energy, and Area Evaluation

Area. Tbl. IV shows the breakdown of the components for MANT and other accelerators, which are all synthesized using a 28 nm process [31], [43]. Note that Olive was originally synthesized with 22 nm process [29]. All accelerators are configured with identical buffer sizes. In addition, their vector units are the same in our evaluation, so the area is not listed in Tbl. IV. In the subsequent evaluation, the number of cores used in the simulator matches those listed in Tbl. IV.

Linear Layer. Fig. 12 presents the performance and energy results of various methods across LLaMA-7B&65B and OPT-

TABLE IV: The area of core components and buffers for MANT and other baseline models using a 28nm process.

Arch.	Core			Others
	Component	Number	Area (mm^2)	
MANT	8-bit PE ($281.75\mu m^2$)	1024	0.302	Buffer: 512KB, 4.2 mm^2
	RQU ($416.63\mu m^2$)	32		
OliVe	4-bit PE ($79.57\mu m^2$)	4096	0.337	Vector Units: #64, 0.069 mm^2
	4-bit Decoder ($48.51\mu m^2$)	128		
	8-bit Decoder ($73.25\mu m^2$)	64		
ANT	4-bit PE ($79.57\mu m^2$)	4096	0.327	Accumulation Units: #32, 0.016 mm^2
	Decoder ($4.9\mu m^2$)	128		
Tender	4-bit PE ($77.28\mu m^2$)	4096	0.317	

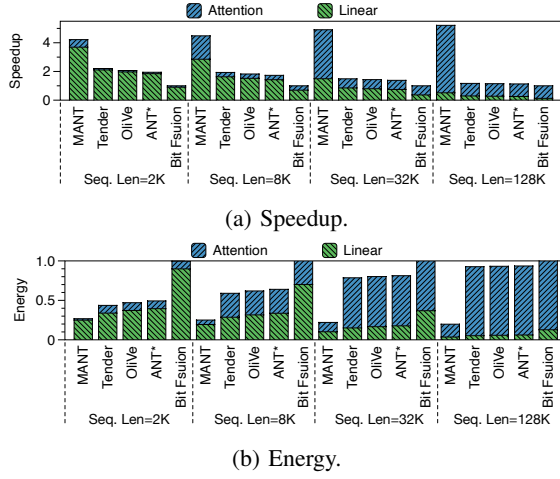


Fig. 13: (a) The speedup of all layers on various accelerators. (b) The normalized energy of all layers on various accelerators.

6.7B&13B. Given the similarity in LLaMA family models, we focus our evaluation on LLaMA-7B and the larger LLaMA-65B. Note that ANT* can not recover the accuracy to align with Tender, OliVe, and MANT. A comparison between OliVe and BitFusion explains the advantages of efficient outlier handling. BitFusion has higher latency and energy because it performs computation in 8 and 16 bits. Tender outperforms Olive because the 8-bit layer is less than OliVe. By leveraging flexible encoding and group-wise quantization, MANT outperforms Tender, OliVe, ANT*, and BitFusion with average speedups of $1.83\times$, $1.96\times$, $2.00\times$, and $4.93\times$, respectively.

Compared to Tender, OliVe, and ANT*, the energy efficiency of MANT mainly comes from static energy, which is related to execution cycles. The reduced bit width in MANT lowers DRAM and buffer energy. The more dequantization in group quantization and the extra shift operation increase the power usage of cores. Thus, MANT has similar core energy to other baselines, even with a lower bit width. In total, MANT achieves energy reductions of 1.39, 1.54, 1.57, and 4.16 times compared to Tender, OliVe, ANT*, and BitFusion, respectively.

A typical of GEMM ($M \times K \times N$) in LLaMA-7B is $(M, 4096) \times (4096, 4096)$. In this case, the non-overlapped quantization overhead (in the last iterations) occupies 0.3%. This ratio is decided by the K and N , as we detail in Sec. VI-E.

Performance and Energy for All Layers. We evaluate the performance and energy of complete layers containing linear and attention. Fig. 13 shows the speedups MANT achieves over other accelerators at various sequence lengths. The evaluation uses the LLaMA-7B model with sequence lengths ranging from 2K to 128K. The baselines do not quantize the attention layer and, therefore, employ 16-bit computation in this layer.

The overall model performance is decided by both linear and attention layers. As the sequence length increases, the attention layer becomes increasingly dominant. With 2K sequence length, the linear layer's latency surpasses that of the attention layer, so the overall speedup and energy reduction are primarily

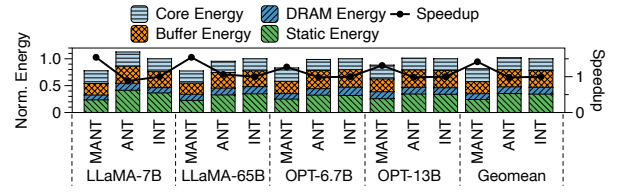


Fig. 14: Speed and energy breakdown in the linear layer. All quantizations use the group size of 64.

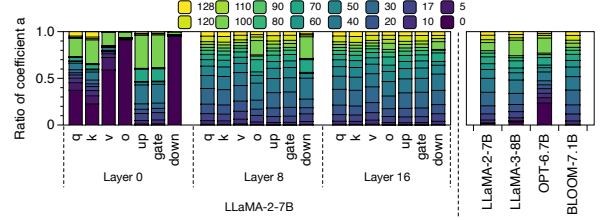


Fig. 15: The data type ratio in various tensor, layer, and models.

determined by the linear layer. However, when the sequence length reaches 128K, the impact of speedup and energy reduction in the linear layer nearly diminishes. With a sequence length of 128K, OliVe achieves only a $1.15\times$ speedup, and Tender achieves a $1.17\times$ speedup over BitFusion. Therefore, at a sequence length of 128K, the improvement of MANT is primarily determined by the attention layer.

MANT consistently delivers speedups between $2.04\text{--}4.54\times$ and energy reductions from $1.76\text{--}4.12\times$ compared to OliVe across different sequence lengths. Moreover, MANT achieves on average $2.99\times$ (up to $4.46\times$) speedup and $2.81\times$ (up to $4.10\times$) energy reduction to Tender in different sequence lengths.

Data Type Ratio. Fig. 15 shows the selection ratio of a in different tensors, layers, and models. The layer₀ of LLaMA-2-7B and OPT-6.7B mostly select $a=0$, while other layers and models have a relatively uniform selection.

D. Comparison and Discussion of Group-wise Methods

Group-wise quantization can significantly improve the results. To discuss the benefits of adaptive data types, we compare the perplexity results with group-wise INT quantization. In addition, we extended ANT and OliVe to support group-wise quantization. For a fair accuracy comparison, we dynamically calculate the per-group scaling factor in activation quantization for all methods. It is worth noting that the other methods do not optimize the process of scaling factor computation, leading to non-negligible quantization overhead. Tbl. V shows the group-wise comparison of 4-bit weight and activation quantization.

ANT faces challenges in group-wise quantization. For weight quantization, ANT can easily select data types from INT, flint, and PoT for each group. However, ANT does not support real-time data type selection for activation quantization. It is difficult to determine data types for each group offline due to variable sequence lengths. Therefore, ANT adaptively selects per-group data types for weight quantization and per-tensor data types for activation, using a per-group scaling factor

TABLE V: W4A4 results in different group sizes. The dataset is Wikitext. The PPL of FP16 is 5.47.

LLaMA-2-7B		MANT	OliVe	ANT	INT	MXFP4
W4A4	G-128	6.26	6.43	6.49	6.54	-
	G-64	5.91	6.31	6.38	6.14	-
	G-32	5.76	6.72	6.23	5.95	7.16

for both weight and activation. Selecting a single data type for the entire tensor is typically unsuitable for each group, making ANT less effective than naive INT when the group size is 64 or 32. OliVe performs better than ANT and INT when the group size is 128. However, OliVe does not benefit from smaller group sizes. Olive sacrifices a victim value to provide more bit width for the outlier value. However, while group-wise quantization mitigates the effect of outliers, as the group size decreases, the negative impact of victims in OliVe outweighs the benefits of protecting outliers.

MANT outperforms group-wise OliVe, ANT, and INT across different group sizes. Besides, we compare the perplexity with MXFP using a group size of 32. The scaling factor with an 8-bit exponent (E8M0) increases the quantization error, raising the PPL to 7.16. The advantage of MXFP is the fast data type casting from FP16 to MXFP.

To isolate the benefits of the MANT data type, we extend ANT architecture to support group quantization. For weights, we determine the data types for each group individually. Since ANT does not support real-time data type selection, we use a calibration dataset to select a global data type for the entire tensor and apply per-group scaling factors for activations. Additionally, we implement group-wise INT for real-time KV cache quantization. To ensure comparable PPL performance with MANT, we use a 4/8 mixed precision configuration for ANT. As illustrated in Fig. 14, with the same group size of 64, MANT achieves an average speedup of $1.70\times$ speedup and $1.55\times$ energy efficiency over ANT.

VIII. CONCLUSION

Our paper introduces a new encoding paradigm called MANT, designed to support various data distributions and heterogeneous computability for group-wise weight quantization. To solve the challenge of employing group-wise quantization in KV cache, we propose real-time quantization components. We extend the processing unit (PE) in a systolic array to support the efficient computation of MANT, and we implement the real-time update components through the lightweight comparator and fixed-point multiplier. Our evaluation shows MANT achieves the minimal accuracy loss in several LLMs and achieves on average $2.99\times$ (up to $4.46\times$) speedup and $2.81\times$ (up to $4.10\times$) energy reduction to the SOTA LLM accelerator.

ACKNOWLEDGMENT

This work was supported by the National Key R&D Program of China under Grant 2022YFB4501400, the National Natural Science Foundation of China (NSFC) grant (62222210, U21B2017 and 62072297). This work was also supported by

Shanghai Qi Zhi Institute Innovation Program SQZ202316. Cong Guo was supported by Shanghai Jiao Tong University Outstanding Doctoral Graduate Development Scholarship. The authors express their gratitude to the anonymous reviewers for their insightful feedback, which greatly contributed to improving this work.

REFERENCES

- [1] M. Adnan, A. Arunkumar, G. Jain, P. J. Nair, I. Soloveychik, and P. Kamath, "Keyformer: KV cache reduction through key tokens selection for efficient generative inference." [Online]. Available: <http://arxiv.org/abs/2403.09054>
- [2] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "Gqa: Training generalized multi-query transformer models from multi-head checkpoints," 2023.
- [3] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, M. Jaggi, D. Alistarh, T. Hoefer, and J. Hensman, "Quarot: Outlier-free 4-bit inference in rotated llms," 2024. [Online]. Available: <https://arxiv.org/abs/2404.00456>
- [4] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, "Longbench: A bilingual, multitask benchmark for long context understanding," 2024. [Online]. Available: <https://arxiv.org/abs/2308.14508>
- [5] A. Bakhoda, G. Yuan, W. Fung, H. Wong, and T. Aamodt, "Analyzing cuda workloads using a detailed gpu simulator," 05 2009, pp. 163 – 174.
- [6] Y. Cai, Z. Yao, Z. Dong, A. Gholami, M. W. Mahoney, and K. Keutzer, "Zeroq: A novel zero shot quantization framework," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 13 169–13 178.
- [7] Z. Cai and N. Vasconcelos, "Rethinking differentiable search for mixed-precision neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 2349–2358.
- [8] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE journal of solid-state circuits*, vol. 52, no. 1, pp. 127–138, 2016.
- [9] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann *et al.*, "Palm: Scaling language modeling with pathways," *arXiv preprint arXiv:2204.02311*, 2022.
- [10] S. Dai, R. Venkatesan, H. Ren, B. Zimmer, W. J. Dally, and B. Khailany, "Vs-quant: Per-vector scaled quantization for accurate low-precision neural network inference," 2021.
- [11] B. Darvish Rouhani, D. Lo, R. Zhao, M. Liu, J. Fowers, K. Ovtcharov, A. Vinogradsky, S. Massengill, L. Yang, R. Bittner, A. Forin, H. Zhu, T. Na, P. Patel, S. Che, L. Chand Koppaka, X. SONG, S. Som, K. Das, S. T. S. Reinhardt, S. Lanka, E. Chung, and D. Burger, "Pushing the limits of narrow precision inferencing at cloud scale with microsoft floating point," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 10 271–10 281.
- [12] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Llm. int8 (): 8-bit matrix multiplication for transformers at scale," *arXiv preprint arXiv:2208.07339*, 2022.
- [13] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," 2023.
- [14] T. Dettmers, R. Svirschevski, V. Egiazarian, D. Kuznedelev, E. Frantar, S. Ashkboos, A. Borzunov, T. Hoefer, and D. Alistarh, "Spqr: A sparse-quantized representation for near-lossless llm weight compression," 2023.
- [15] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [16] H. Duanmu, Z. Yuan, X. Li, J. Duan, X. Zhang, and D. Lin, "Skvq: Sliding-window key and value cache quantization for large language models," 2024. [Online]. Available: <https://arxiv.org/abs/2405.06219>
- [17] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, *et al.*, "The llama 3 herd of models," 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>

- [18] E. Frantar and D. Alistarh, "Sparsegpt: Massive language models can be accurately pruned in one-shot," 2023. [Online]. Available: <https://arxiv.org/abs/2301.00774>
- [19] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," 2023.
- [20] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, S. Presser, and C. Leahy, "The pile: An 800gb dataset of diverse text for language modeling," 2020.
- [21] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac'h, H. Li, K. McDonnell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, "A framework for few-shot language model evaluation," 12 2023. [Online]. Available: <https://zenodo.org/records/10256836>
- [22] Y. Guan, J. Leng, C. Li, Q. Chen, and M. Guo, "How far does bert look at: Distance-based clustering and analysis of bert's attention," *arXiv preprint arXiv:2011.00943*, 2020.
- [23] Y. Guan, Z. Li, J. Leng, Z. Lin, and M. Guo, "Transkimmer: Transformer learns to layer-wise skim," *arXiv preprint arXiv:2205.07324*, 2022.
- [24] Y. Guan, Z. Li, Z. Lin, Y. Zhu, J. Leng, and M. Guo, "Block-skim: Efficient question answering for transformer," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 10, 2022, pp. 10710–10719.
- [25] Y. Guan, C. Yu, Y. Zhou, J. Leng, C. Li, and M. Guo, "Fractal: Joint multi-level sparse pattern tuning of accuracy and performance for dnn pruning," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, New York, NY, USA, 2024, p. 416–430.
- [26] C. Guo, F. Cheng, Z. Du, J. Kiessling, J. Ku, S. Li, Z. Li, M. Ma, T. Molom-Ochir, B. Morris *et al.*, "A survey: Collaborative hardware and software design in the era of large language models," *arXiv preprint arXiv:2410.07265*, 2024.
- [27] C. Guo, B. Y. Hsueh, J. Leng, Y. Qiu, Y. Guan, Z. Wang, X. Jia, X. Li, M. Guo, and Y. Zhu, "Accelerating sparse dnn models without hardware-support via tile-wise sparsity," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–15.
- [28] C. Guo, Y. Qiu, J. Leng, X. Gao, C. Zhang, Y. Liu, F. Yang, Y. Zhu, and M. Guo, "SQuant: On-the-fly data-free quantization via diagonal hessian approximation," in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=JXhR0KNZzOc>
- [29] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y. Liu, M. Guo, and Y. Zhu, "Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, New York, NY, USA, 2023, pp. 1–15.
- [30] C. Guo, F. Xue, J. Leng, Y. Qiu, Y. Guan, W. Cui, Q. Chen, and M. Guo, "Accelerating sparse dnns based on tiled gemm," *IEEE Transactions on Computers*, 2024.
- [31] C. Guo, C. Zhang, J. Leng, Z. Liu, F. Yang, Y. Liu, M. Guo, and Y. Zhu, "Ant: Exploiting adaptive numerical data type for low-bit deep neural network quantization," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1414–1433.
- [32] C. Guo, R. Zhang, J. Xu, J. Leng, Z. Liu, Z. Huang, M. Guo, H. Wu, S. Zhao, J. Zhao, and K. Zhang, "Gmlake: Efficient and transparent gpu memory defragmentation for large-scale dnn training with virtual memory stitching," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, New York, NY, USA, 2024, p. 450–466.
- [33] C. Guo, Y. Zhou, J. Leng, Y. Zhu, Z. Du, Q. Chen, C. Li, B. Yao, and M. Guo, "Balancing Efficiency and Flexibility for DNN Acceleration via Temporal GPU-Systolic Array Integration," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–6.
- [34] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," *arXiv preprint arXiv:1510.00149*, 2015.
- [35] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "Kvquant: Towards 10 million context length llm inference with kv cache quantization," 2024.
- [36] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer, "Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension," 2017. [Online]. Available: <https://arxiv.org/abs/1705.03551>
- [37] N. P. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. Patterson, "Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," 2023.
- [38] H. Kang, Q. Zhang, S. Kundu, G. Jeong, Z. Liu, T. Krishna, and T. Zhao, "Gear: An efficient kv cache compression recipe for near-lossless generative inference of llm," 2024.
- [39] M. Khairy, Z. Shen, T. Aamodt, and T. Rogers, "Accel-sim: An extensible simulation framework for validated gpu modeling," 05 2020, pp. 473–486.
- [40] S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney, and K. Keutzer, "Squeezellm: Dense-and-sparse quantization," 2023.
- [41] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," 2023.
- [42] Y. LeCun, J. Denker, and S. Solla, "Optimal brain damage," in *Advances in Neural Information Processing Systems*, D. Touretzky, Ed., vol. 2. Morgan-Kaufmann, 1989.
- [43] J. Lee, W. Lee, and J. Sim, "Tender: Accelerating large language models via tensor decomposition and runtime requantization," 2024. [Online]. Available: <https://arxiv.org/abs/2406.12930>
- [44] J. Leng, T. Hetherington, A. ElTantawy, S. Gilani, N. Kim, T. Aamodt, and V. Janapa Reddi, "Gpuwatch: enabling energy optimizations in gpgpus," *ACM SIGARCH Computer Architecture News*, vol. 41, 07 2013.
- [45] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, "SnapKV: LLM knows what you are looking for before generation." [Online]. Available: <http://arxiv.org/abs/2404.14469>
- [46] Z. Li, C. Guo, Z. Zhu, Y. Zhou, Y. Qiu, X. Gao, J. Leng, and M. Guo, "Efficient adaptive activation rounding for post-training quantization," 2023. [Online]. Available: <https://arxiv.org/abs/2208.11945>
- [47] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for llm compression and acceleration," 2023.
- [48] S. Lin, J. Hilton, and O. Evans, "Truthfulqa: Measuring how models mimic human falsehoods," 2022. [Online]. Available: <https://arxiv.org/abs/2109.07958>
- [49] Z. Liu, J. Leng, Z. Zhang, Q. Chen, C. Li, and M. Guo, "VELTAIR: towards high-performance multi-tenant deep learning services via adaptive compilation and scheduling," in *ASPLOS '22: 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Lausanne, Switzerland, 28 February 2022 - 4 March 2022*, B. Falsafi, M. Ferdman, S. Lu, and T. F. Wenisch, Eds. ACM, 2022, pp. 388–401. [Online]. Available: <https://doi.org/10.1145/3503222.3507752>
- [50] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "Kivi: A tuning-free asymmetric 2bit quantization for kv cache," *arXiv preprint arXiv:2402.02750*, 2024.
- [51] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," 2016.
- [52] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," in *International Conference on Learning Representations*, 2018.
- [53] D. Miyashita, E. H. Lee, and B. Murmann, "Convolutional neural networks using logarithmic data representation," *arXiv preprint arXiv:1603.01025*, 2016.
- [54] N. Muralimanohar, R. Balasubramanian, and N. P. Jouppi, "Cacti 6.0: A tool to model large caches," *HP laboratories*, vol. 27, p. 28, 2009.
- [55] W. Niu, J. Guan, Y. Wang, G. Agrawal, and B. Ren, "Dnnfusion: accelerating deep neural networks execution with advanced operator fusion," ser. PLDI 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 883–898. [Online]. Available: <https://doi.org/10.1145/3453483.3454083>
- [56] Nvidia, "Nvidia tesla v100 gpu architecture," in *Technical report*. NVIDIA, 2017.
- [57] Nvidia, "Nvidia a100 tensor core architecture," in *Technical report*. NVIDIA, 2020.
- [58] Nvidia, "Nvidia h100 tensor core gpu architecture," in *Technical report*. NVIDIA, 2022.
- [59] Y. Qiu, J. Leng, C. Guo, Q. Chen, C. Li, M. Guo, and Y. Zhu, "Adversarial defense through network profiling based path extraction," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4772–4781.
- [60] Z. Qu, L. Liu, F. Tu, Z. Chen, Y. Ding, and Y. Xie, "Dota: detect and omit weak attentions for scalable transformer acceleration," ser. ASPLOS

- '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 14–26. [Online]. Available: <https://doi.org/10.1145/3503222.3507738>
- [61] A. Ramachandran, Z. Wan, G. Jeong, J. Gustafson, and T. Krishna, “Algorithm-hardware co-design of distribution-aware logarithmic-posit encodings for efficient dnn inference,” 2024.
- [62] E. Reggiani, A. Pappalardo, M. Doblas, M. Moreto, M. Olivieri, O. S. Unsal, and A. Cristal, “Mix-gemm: An efficient hw-sw architecture for mixed-precision quantized deep neural networks inference on edge devices,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 1085–1098.
- [63] B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, S. Dusan, V. Elango, M. Golub, A. Heinecke, P. James-Roxby, D. Jani, G. Kolhe, M. Langhammer, A. Li, L. Melnick, M. Mesmakhosroshahi, A. Rodriguez, M. Schulte, R. Shafipour, L. Shao, M. Siu, P. Dubey, P. Micikevicius, M. Naumov, C. Verrilli, R. Wittig, D. Burger, and E. Chung, “Microscaling data formats for deep learning,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.10537>
- [64] T. L. Scao, A. Fan, C. Akiki, E. Pavlick, S. Ilić, D. Hesslow, R. Castagné, A. S. Luccioni, F. Yvon, M. Gallé *et al.*, “Bloom: A 176b-parameter open-access multilingual language model,” *arXiv preprint arXiv:2211.05100*, 2022.
- [65] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo, “Omniquant: Omnidirectionally calibrated quantization for large language models,” 2024.
- [66] H. Sharma, J. Park, D. Mahajan, E. Amaro, J. K. Kim, C. Shao, A. Mishra, and H. Esmailzadeh, “From high-level deep neural models to fpgas,” in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–12.
- [67] H. Sharma, J. Park, N. Suda, L. Lai, B. Chau, V. Chandra, and H. Esmailzadeh, “Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural network,” in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 764–775.
- [68] S. Shen, Z. Dong, J. Ye, L. Ma, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, “Q-bert: Hessian based ultra low precision quantization of bert,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 05, 2020, pp. 8815–8821.
- [69] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, D. Y. Fu, Z. Xie, B. Chen, C. Barrett, J. E. Gonzalez, P. Liang, C. Ré, I. Stoica, and C. Zhang, “Flexgen: High-throughput generative inference of large language models with a single gpu,” 2023.
- [70] Z. Song, B. Fu, F. Wu, Z. Jiang, L. Jiang, N. Jing, and X. Liang, “Drq: dynamic region-based quantization for deep neural network acceleration,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 1010–1021.
- [71] H. Sun, Z. Chen, X. Yang, Y. Tian, and B. Chen, “TriForce: Lossless acceleration of long sequence generation with hierarchical speculative decoding,” [Online]. Available: <http://arxiv.org/abs/2404.11912>
- [72] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, “Llama: Open and efficient foundation language models,” 2023.
- [73] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, R. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, “Llama 2: Open foundation and fine-tuned chat models,” 2023.
- [74] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. D. Sa, “QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.04396>
- [75] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [76] Y. Wang, C. Zhang, Z. Xie, C. Guo, Y. Liu, and J. Leng, “Dual-side sparse tensor core,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 1083–1095.
- [77] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “Smoothquant: Accurate and efficient post-training quantization for large language models,” 2023.
- [78] J. Xu, R. Zhang, C. Guo, W. Hu, Z. Liu, F. Wu, Y. Feng, S. Sun, C. Shao, Y. Guo, J. Zhao, K. Zhang, M. Guo, and J. Leng, “vtensor: Flexible virtual tensor management for efficient llm serving,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.15309>
- [79] Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, “Zeroquant: Efficient and affordable post-training quantization for large-scale transformers,” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 27 168–27 183. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/ad7f7fa39d65e2983d724ff7da57f00ac-Paper-Conference.pdf
- [80] Z. Yuan, Y. Shang, Y. Zhou, Z. Dong, C. Xue, B. Wu, Z. Li, Q. Gu, Y. J. Lee, Y. Yan, B. Chen, G. Sun, and K. Keutzer, “Llm inference unveiled: Survey and rooftop model insights,” 2024.
- [81] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, “Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference,” in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 811–824.
- [82] A. H. Zadeh, M. Mahmoud, A. Abdelhadi, and A. Moshovos, “Mokey: enabling narrow fixed-point inference for out-of-the-box floating-point transformer models,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, New York, NY, USA, 2022, pp. 888–901.
- [83] C. Zhang, Y. Wang, Z. Xie, C. Guo, Y. Liu, J. Leng, G. Sun, Z. Ji, R. Wang, Y. Xie *et al.*, “Dstc: Dual-side sparsity tensor core for dnn acceleration on modern gpu architectures,” *IEEE Transactions on Computers*, 2024.
- [84] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, “Opt: Open pre-trained transformer language models,” *arXiv preprint arXiv:2205.01068*, 2022.
- [85] Z. Zhang, S. Liu, R. Chen, B. Kailkhura, B. Chen, and Z. Wang, “Q-hitter: A better token oracle for efficient LLM inference via sparse-quantized KV cache,”
- [86] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, “H₂S₂O: Heavy-hitter oracle for efficient generative inference of large language models,” [Online]. Available: <http://arxiv.org/abs/2306.14048>
- [87] H. Zhao, W. Cui, Q. Chen, Y. Zhang, Y. Lu, C. Li, J. Leng, and M. Guo, “Tacker: Tensor-cuda core kernel fusion for improving the gpu utilization while ensuring qos,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 800–813.
- [88] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasicki, “Atom: Low-bit quantization for efficient and accurate llm serving,” 2023.
- [89] S. Zheng, S. Chen, P. Song, R. Chen, X. Li, S. Yan, D. Lin, J. Leng, and Y. Liang, “Chimera: An analytical optimizing framework for effective compute-intensive operators fusion,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 1113–1126.
- [90] A. Zhou, A. Yao, Y. Guo, L. Xu, and Y. Chen, “Incremental network quantization: Towards lossless cnns with low-precision weights,” *arXiv preprint arXiv:1702.03044*, 2017.
- [91] S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, and Y. Zou, “Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients,” *arXiv preprint arXiv:1606.06160*, 2016.
- [92] Y. Zhou, J. Leng, Y. Song, S. Lu, M. Wang, C. Li, M. Guo, W. Shen, Y. Li, W. Lin, X. Liu, and H. Wu, “ugrapher: High-performance graph operator computation via unified abstraction for graph neural networks,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 878–891. [Online]. Available: <https://doi.org/10.1145/3575693.3575723>
- [93] Y. Zhou, M. Yang, C. Guo, J. Leng, Y. Liang, Q. Chen, M. Guo, and Y. Zhu, “Characterizing and demystifying the implicit convolution algorithm on commercial matrix-multiplication accelerators,” in *2021 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2021, pp. 214–225.